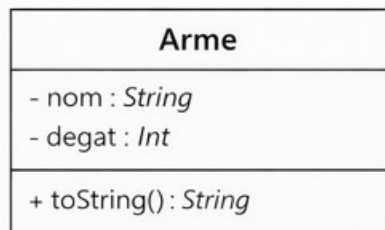


Projet : Kotlin P00

Partie 1 : Les bases de la P00 et l'encapsulation

I. La classe Arme

1. Implémentation de la classe



Créer une classe Arme avec :

- Deux propriétés :

- **nom** : le nom de l'arme (String – non modifiable)
- **degat** : les dégâts de l'arme (Int – modifiable)

Vous devez implémenter un *setter personnalisé* pour cet attribut afin de :

- ▶ vérifier que la nouvelle valeur est strictement positive ;
- ▶ mettre à jour la valeur uniquement si la condition est respectée et afficher le message suivant:
« **Modification des dégâts de l'arme NomArme.** »
- ▶ si la condition n'est pas respectée afficher le message suivant: « **Valeur invalide pour les dégâts** »

- Une méthode :

- **toString()** pour que l'affichage d'une arme soit sous la forme :
(NomArme, Degat)

2. Tests de la classe

- Créer une arme Épée avec 15 dégâts puis afficher ses caractéristiques avec la méthode toString().

Résultat attendu :

(Épée, 15)

- Créer une arme Arc avec 10 dégâts puis afficher ses caractéristiques avec la méthode toString().

Résultat attendu :

(Arc, 10)

- Modifier les dégâts de l'Arc à 12 puis afficher ses caractéristiques avec la méthode toString().

Résultat attendu :

Modification des dégâts de l'arme Arc.

(Arc, 12)

- Modifier les dégâts de l'Arc à -5

Résultat attendu :

Valeur invalide pour les dégâts

II. La classe Joueur

1. Implémentation de la classe

Joueur
- nom : <i>String</i> - vie : <i>Int</i> - armeEquipee : <i>Arme</i> - inventaire : <i>MutableList<Arme></i>
+ attaquer() : <i>Int</i> + recevoirDegats(<i>degat</i> : <i>Int</i>) + ajouterArme(<i>arme</i> : <i>Arme</i>) + changerArme(<i>nomArme</i> : <i>String</i>) + afficherInventaire() + toString() : <i>String</i>

Créer une classe Joueur avec :

- Quatre propriétés :

- **nom** : le nom du joueur (*String* – non modifiable)
- **vie** : points de vie du joueur (*Int* – modifiable)
- **armeEquipee** : l'arme actuellement équipée (*Arme* – modifiable).
- **inventaire** : une liste d'armes que le joueur possède (*MutableList<Arme>*)

- Deux constructeurs :

- **constructeur primaire** : permet de créer un objet Joueur avec une arme équipée et l'arme : **Arme(Poing, 5)** est présente par défaut dans l'inventaire.
- **constructeur secondaire** : permet de créer un joueur sans arme équipée. Dans ce cas, l'arme **Arme(Poing, 5)** est l'arme équipée par défaut et l'inventaire est vide.

L'arme équipée n'est pas forcément présente dans l'inventaire.

- Six méthodes :

- **attaquer()** retourne les dégâts de l'arme équipée.
- **recevoirDegats(degat: Int)** diminue les points de vie du joueur avec les dégâts passé en paramètre. Attention, les points de vie ne peuvent pas être négatifs (minimum 0).
- **ajouterArme(arme:Arme)** ajoute une arme dans l'inventaire
- **changerArme(nomArme: String)** équipe une arme depuis l'inventaire et remet l'arme précédente dans l'inventaire. (aide : voir les méthodes `.add()`, `.remove()` et `.find()` sur les listes en Kotlin)
- **afficherInventaire()** affiche toutes les armes de l'inventaire
- **toString()** affichage d'un joueur soit sous la forme : **(Nom, vie=Vie, arme=ArmeEquipee, inventaire=ListeArmes)**

2. Tests de la classe

- Créer le joueur Arthur avec 50 points de vie et équipé de l'Épée.

Afficher les caractéristiques du joueur Arthur avec la méthode `toString()`.

Résultat attendu :

(Arthur, vie=50, arme=(Épée, 15), inventaire=[(Poing, 5)])

- Créer une nouvelle arme : une Hache avec 20 de dégâts.

Ajouter la Hache et l'Arc dans l'inventaire d'Arthur.

Afficher l'inventaire avec la méthode `afficherInventaire()`.

Résultat attendu :

Inventaire de Arthur :

(Poing, 5)

(Arc, 12)

(Hache, 20)

- Changer l'arme d'Arthur en équipant la Hache.

Résultat attendu :

Arthur a équipé (Hache, 20)

- Changer l'arme d'Arthur avec une arme inexistante (ex : Lance)

Résultat attendu :

Arme non trouvée dans l'inventaire

- Créer un joueur Lancelot avec 50 points de vie et sans arme équipée.
- Afficher les caractéristiques de ce joueur avec la méthode toString().

Résultat attendu :

(Lancelot, vie=50, arme=(Poing, 5), inventaire=[])

- Équiper le joueur Lancelot avec l'arme Arc.
- Afficher l'arme équipé de Lancelot dans un string.

Résultat attendu :

Lancelot est équipé de l'arme : (Arc, 12)

- Afficher l'inventaire mis à jour de Lancelot avec la méthode afficherInventaire().

Résultat attendu :

Inventaire de Lancelot :

(Poing, 5)

III. La classe Combat

1. Implémentation de la classe

Combat
- joueur1 : Joueur - joueur2 : Joueur
- jouerTour(attaquant : Joueur, defenseur : Joueur) - afficherVainqueur() + lancer()

Créer une classe Combat avec :

- Deux propriétés :

- **joueur1** : le premier joueur (Joueur – non modifiable).
- **joueur2** : le second joueur (Joueur – non modifiable).

Ces deux propriétés doivent être accessible uniquement dans la classe Combat.

- Trois méthodes :

- **jouerTour(attaquant: Joueur, defenseur: Joueur)** une méthode accessible uniquement dans la classe Combat.

Cette méthode permet à un joueur attaquant d'appliquer les dégâts de son arme sur la vie d'un joueur défenseur. Les messages suivants s'affichent :

***NomAttaquant* attaque *NomDefenseur* (degat dégâts)**

***NomDefenseur* a VieDefenseur PV**

- **afficherVainqueur()** une méthode accessible uniquement dans la classe Combat.

Cette méthode permet d'afficher **Fin du combat !** puis le nom du joueur gagnant ***NomGagnant* a gagné !**

OU Match nul !

- **lancer()** une méthode accessible partout.

Cette méthode permet qu'à chaque tour, le joueur 1 attaque en premier, puis le joueur 2 riposte. Cette alternance continue jusqu'à ce qu'un joueur ait 0 PV.

Cette méthode affiche au début du combat :

Début du combat entre *NomJoueur1* et *NomJoueur2* !

Le numéro du tour est affiché au début de chaque tour.

--- Tour XX ---

Dans cette méthode vous devez appeler les méthodes internes à la classe : **jouerTour(attaquant: Joueur, défenseur: Joueur)** et **afficherVainqueur()**

2. Tests de la classe

- Lancer un combat entre Arthur et Lancelot.

Résultat attendu :

Début du combat entre Arthur et Lancelot !

--- Tour 1 ---

Arthur attaque Lancelot (20 dégâts)

Lancelot a 30 PV

Lancelot attaque Arthur (12 dégâts)

Arthur a 38 PV

--- Tour 2 ---

Arthur attaque Lancelot (20 dégâts)

Lancelot a 10 PV

Lancelot attaque Arthur (12 dégâts)

Arthur a 26 PV

--- Tour 3 ---

Arthur attaque Lancelot (20 dégâts)

Lancelot a 0 PV

Fin du combat !

Arthur a gagné !

IV. Pour aller plus loin (partie non obligatoire)

Il existe certaines incohérences ou bug possible dans le code proposé dans cette partie.

Proposer une solution pour résoudre les problème suivants :

- Incohérence sur l'inventaire

Les deux constructeurs gèrent l'inventaire de manière contradictoire. L'arme équipée n'est pas forcément dans l'inventaire mais cela n'est pas cohérent dans le contexte du jeu.

- Il est possible de créer un objet Arme avec des dégâts négatifs.
- Il est possible de créer un objet Joueur avec une vie négative.
- Le combat n'est très répétitif. Ajouter un coup critique aléatoire dans le combat ou une autre fonctionnalité.
- Il est possible d'avoir des doublons dans l'inventaire.

Partie 2 : L'héritage et le polymorphisme

I. La classe Personnage (classe mère)

1. Implémentation de la classe

Personnage
- nom : <i>String</i> - vie : <i>Int</i> - attaque : <i>Int</i>
+ subirDegats(<i>degats</i> : <i>Int</i>) + attaquer(<i>cible</i> : <i>Personnage</i>) + afficherEtat()

Créer une classe Personnage avec :

- Trois propriétés :

- **nom** : le nom du personnage (String – non modifiable)
- **vie** : les points de vie du personnage (Int – modifiable)
- **attaque** : les points d'attaque du personnage (Int – non modifiable)

- Trois méthodes :

- **subirDegats(degats: Int)** : méthode permettant de diminuer la vie du personnage en fonction des dégâts reçus. Puis, affiche le message suivant :

NomPersonnage perd X points de vie (Vie restante : Y)

- **attaquer(cible: Personnage)** : méthode permettant d'attaquer un autre personnage.

Vous devez vérifier que la vie du personnage qui attaque est bien positive. Si ce n'est pas le cas alors afficher : **NomPersonnage ne peut pas attaquer (KO)**

Si la vie de l'attaquant est bien positive, afficher :
NomPersonnage attaque NomCible et inflige X dégâts
Puis, appliquer les dégât sur le personnage cible.

- **afficherEtat()** : cette méthode permet d'afficher les caractéristiques du personnage sous la forme :
Nom : Nom | Vie : Vie | Attaque : Attaque

2. Tests de la classe

- Créer un personnage nommé Perceval avec 100 points de vie et 15 points de dégât.

Créer le personnage nommé Mordred avec 80 points de vie et 20 points de dégât.

Afficher les caractéristiques du personnage nommé Perceval.

Résultat attendu :

Nom : Perceval | Vie : 100 | Attaque : 15

- Lancer une attaque de Mordred contre Perceval.

Résultat attendu :

Mordred attaque Perceval et inflige 20 dégâts

Perceval perd 20 points de vie (Vie restante : 80)

- Tester la méthode subirDegats avec Perceval en infligeant 100 points de dégâts.

Résultat attendu :

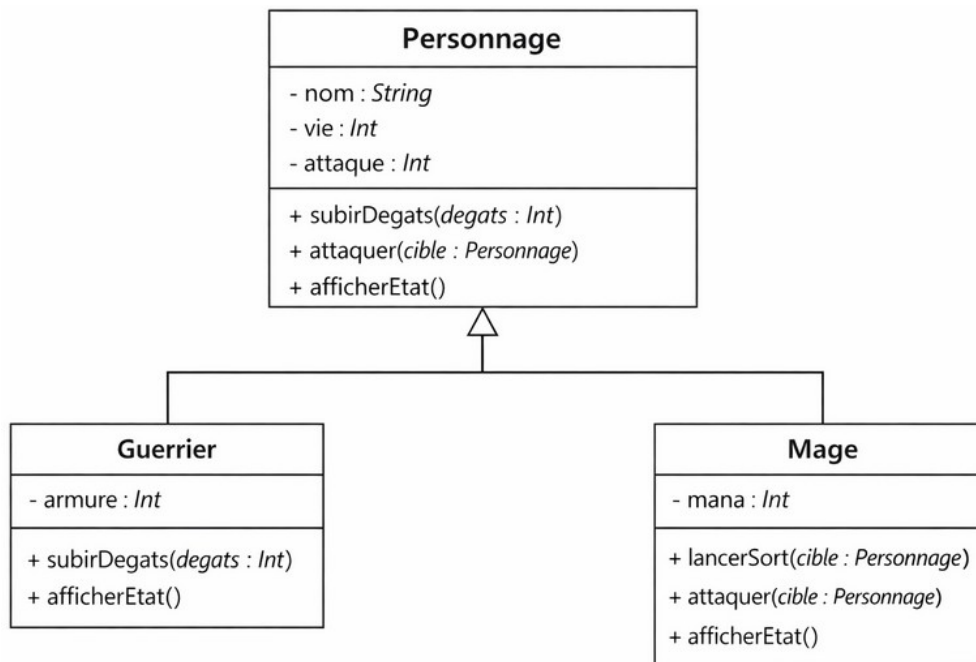
Perceval perd 100 points de vie (Vie restante : -20)

- Lancer une attaque de Perceval contre Mordred.

Résultat attendu :

Perceval ne peut pas attaquer (K0)

II. Les classes filles



1. La classe Guerrier

Créer une classe Guerrier avec :

- Quatre propriétés :

- **nom** : propriété héritée de la classe mère `Personnage()`
- **vie** : propriété héritée de la classe mère `Personnage()`
- **attaque**: propriété héritée de la classe mère `Personnage()`
- **armure**: les points d'armure du guerrier (`Int`, modifiable)

- Trois méthodes :

- **subirDegats(degats: Int)** : méthode héritée de la classe mère `Personnage()` mais redéfinie dans cette classe.

Cette méthode permet de diminuer la vie du personnage en fonction des dégâts réels reçus. Les dégâts réels reçus sont calculés en diminuant les dégâts du nombre de points d'armure du guerrier.

Puis, affiche le message suivant :

NomGuerrier subit **DegatsReels** dégâts avec l'armure (**Vie restante : VieGuerrier**)

- **attaquer(cible: Personnage)** : méthode héritée de la classe mère Personnage()
- **afficherEtat()** : méthode héritée de la classe mère Personnage() mais redéfinie dans cette classe.

Cette méthode permet d'afficher les caractéristiques du Guerrier sous la forme :

Nom : *Nom* | Vie : *Vie* | Attaque : *Attaque* | Armure : *Armure*

2. La classe Mage

Créer une classe Mage avec :

- Deux constructeurs :
 - **constructeur primaire** : permet de créer un objet Mage en renseignant des valeurs pour toutes les propriétés.
 - **constructeur secondaire** : permet de créer un objet Mage en renseignant uniquement le nom du mage. Dans ce cas, le mage a par défaut 100 points de vie, 20 points d'attaque et 50 points de mana.
- Quatre propriétés :
 - **nom** : propriété héritée de la classe mère Personnage()
 - **vie** : propriété héritée de la classe mère Personnage()
 - **attaque**: propriété héritée de la classe mère Personnage()
 - **mana**: les points de mana du mage (Int - modifiable)

- Quatre méthodes :

- **lancerSort(cible : Personnage)** : méthode permettant de lancer une attaque spéciale si le mage a du mana.

Si le mage a au moins 10 points de mana alors le mage peut lancer l'attaque spéciale en infligeant le double de dégâts d'attaque sur la cible. Cette attaque retire 10 points de mana au mage qui lance le sort et affiche :

NomMage lance un sort sur NomCible (Mana restant : ManaMage)

Si le mage n'a pas assez de mana alors cette méthode affiche : **NomMage n'a pas assez de mana, attaque normale**
Ensuite, cette méthode appelle la méthode **attaquer(cible : Personnage)** de la classe mère **Personnage**

- **attaquer(cible: Personnage)** : méthode héritée de la classe mère **Personnage()** mais redéfinie dans cette classe
Cette méthode appelle la méthode **lancerSort(cible : Personnage)**

- **subirDegats(degats: Int)** : méthode héritée de la classe mère **Personnage()**

- **afficherEtat()** : méthode héritée de la classe mère **Personnage()** mais redéfinie dans cette classe.

Cette méthode permet d'afficher les caractéristiques du Guerrier sous la forme :

Nom : Nom | Vie : Vie | Attaque : Attaque | Mana : Mana

3. Tests des classes filles

- Créer trois objets :
 - un guerrier nommé Arthur avec 150 points de vie, 50 points d'attaque et 5 points d'armure.
 - un mage nommé Merlin avec 150 points de vie, 20 points d'attaque et 20 points de mana
 - un mage nommé Morgane (utiliser le constructeur secondaire de la classe Mage)
- Créer une liste mutable d'éléments de type Personnage puis ajouter les personnages Arthur et Merlin à cette liste.

A l'aide d'une boucle for, parcourir la liste de vos personnages afin d'afficher leurs caractéristiques.

Résultat attendu :

Nom : Arthur | Vie : 150 | Attaque : 50 | Armure : 5

Nom : Merlin | Vie : 150 | Attaque : 20 | Mana : 20

- A l'aide d'une boucle, simuler un combat entre Merlin et Arthur en trois tours

Résultat attendu :

--- Tour 1 ---

Merlin lance un sort sur Arthur (Mana restant : 10)

Arthur subit 35 dégâts avec l'armure (Vie restante : 115)

Arthur attaque Merlin et inflige 50 dégâts

Merlin perd 50 points de vie (Vie restante : 100)

--- Tour 2 ---

Merlin lance un sort sur Arthur (Mana restant : 0)

Arthur subit 35 dégâts avec l'armure (Vie restante : 80)

Arthur attaque Merlin et inflige 50 dégâts

Merlin perd 50 points de vie (Vie restante : 50)

--- Tour 3 ---

Merlin n'a pas assez de mana, attaque normale

Merlin attaque Arthur et inflige 20 dégâts

Arthur subit 15 dégâts avec l'armure (Vie restante : 65)

Arthur attaque Merlin et inflige 50 dégâts

Merlin perd 50 points de vie (Vie restante : 0)

- Afficher les caractéristiques des personnages Arthur et Merlin.

Résultat attendu :

Nom : Arthur | Vie : 65 | Attaque : 50 | Armure : 5

Nom : Merlin | Vie : 0 | Attaque : 20 | Mana : 0

III. Pour aller plus loin (partie non obligatoire)

Il existe certaines incohérences ou bug possible dans le code proposé dans cette partie.

Proposer une solution pour résoudre les problèmes suivants :

- Les points de vie des personnages peuvent être négatifs.
- Le calcul des dégâts du guerrier peut poser des problèmes : si l'armure est supérieure aux dégâts, on obtient des dégâts négatifs (ce qui soigne le personnage).
- Dans cette partie il manque une bonne encapsulation des propriétés et des méthodes.
- Le jeu est très répétitif, ajouter des méthodes aux différents personnages pour varier les possibilités lors d'un combat.

Partie 3 : Bilan

énoncé en cours d'écriture

objectif = réunir les deux premières parties.

A rendre =

- compte-rendu contenant ...
- code à rendre...